COL1101 Lecture 20 2026/09/08

Topic: Heap Operations

Announcements

Recap

Binary Heaps

Definition: Let $(S, \leq)$ be an ordered set. A binary tree T with keys from S is called a binary min-heap if it satisfies the following two properties.

- [Structure] T is an ACBT
- [Order] For every node v of T which is not the root of T, we have: $v.parent.key \leq v.key$.

List representation: LOT of an ACBT in a list/array.

HeapUp (u)

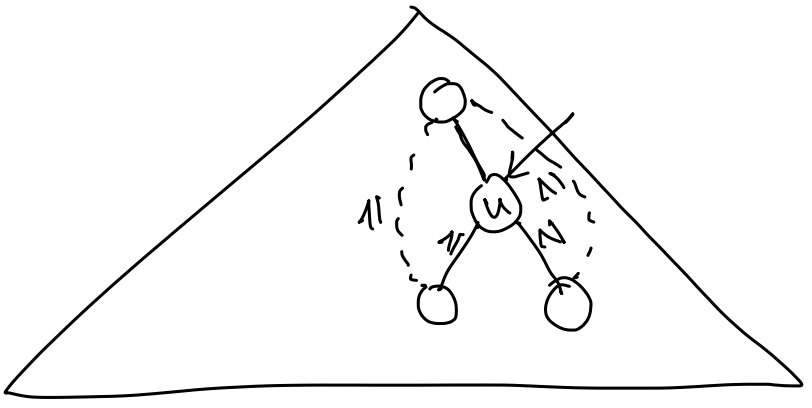
Pre-conditions:

① Almost-complete binary tree.

② $\forall$ nodes $v \neq u$: $v.parent.key \leq v.key$.

③ Keys of u's children $\geq u.parent.key$.

Post-condition : Heap



HeapUp (u):

$u' \leftarrow u$

while $u'.parent.key > u'.key$:

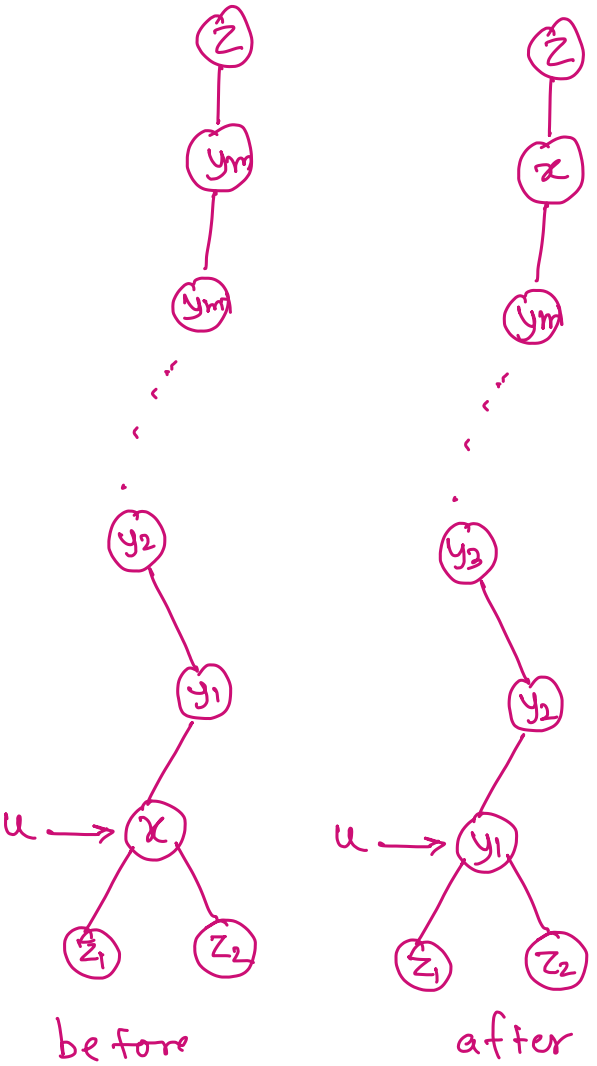
    Exchange keys of
        u' and $u'.parent$.

    $u' \leftarrow u'.parent$.

Running time : $O(depth(u))$

$z_1 \geq y_1$ $\quad y_1 \geq y_2 \geq \cdots \geq y_m \geq x$

$z_2 \geq y_1$ $\quad x \geq z$



Correctness of HeapUp

$z_1 \geq y_1 \quad y_1 \geq y_2 \geq \cdots \geq y_m \geq x \quad x \geq z$

$z_2 \geq y_1$

Keys whose parent's key got changed in the process.

$z_1, z_2, y_m, x, w_1 \ldots w_m$.

$z_1 \geq y_1 \quad y_m \geq x$ (o/w we wouldn't have swapped them)

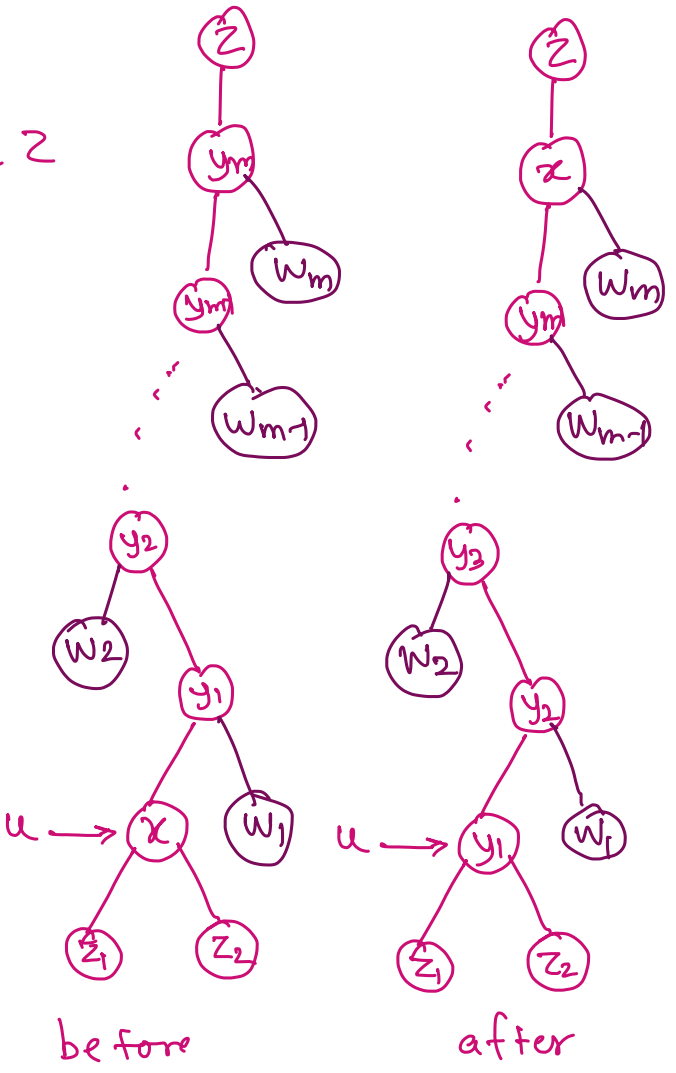
$z_2 \geq y_1$

$x \geq z$ because the alg terminated here

$w_i \geq y_i \geq y_{i+1} \therefore w_i \geq y_{i+1}$

precond precond

$w_m \geq y_m \geq x \quad \therefore w_m \geq x$



Enqueue (x):

1. Attach a new node u with key x at the leftmost available place in the bottommost layer.

   (Create a new layer if needed)

   (ie. append (x) in the array / list representation.)

2. Heapup (u).

Running time : $O(\text{height}) = O(\log_2 n)$

where n = # elements in the heap.

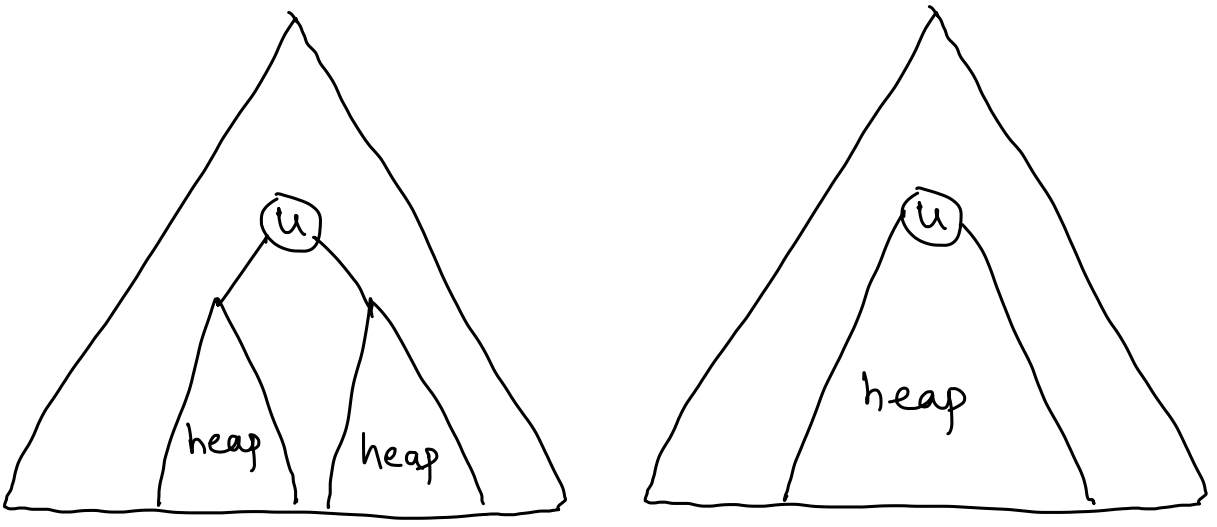
Heap Down (u)

Preconditions :

① Almost complete binary tree

② Left and right subtrees of u are both heaps.

Post condition : Subtree rooted at u is a heap.



Heap Down (u)

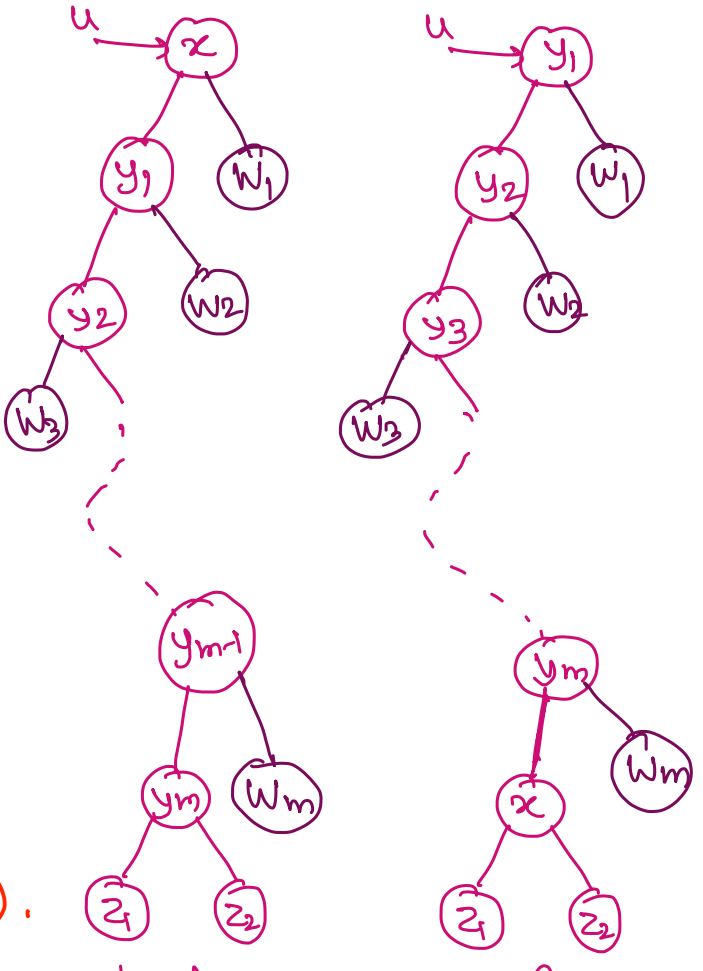
u' ← u

while (u' has a child with key < u'.key):

  v ← child of u' with smaller key.

  Swap u'.key, v.key

  u' ← v

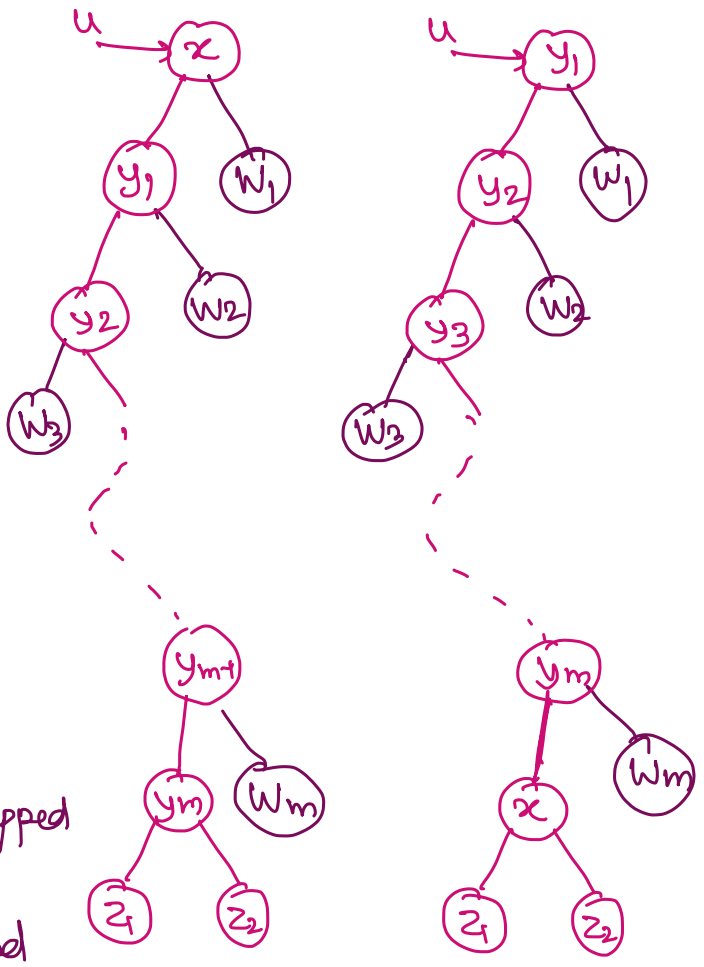


Running time :

O(height of subtree rooted at u).

$y_1 \le y_2 \le \cdots \le y_m \le x$ $\quad x \le z_1$ $\quad x \le z_2$

Correctness of Heap Down



$y_1 \le y_2 \le \cdots \le y_m \le x$ $\quad x \le z_1$, $x \le z_2$

$y_1 \le w_1 \quad y_2 \le w_2 \quad \cdots \quad y_m \le w_m$.

Keys whose parent's key got changed: $z_1, z_2, y_1, x, w_1, \ldots, w_m$.

$z_1 \ge x$, $z_2 \ge x$ } because the alg didn't swap x with z_1, z_2.

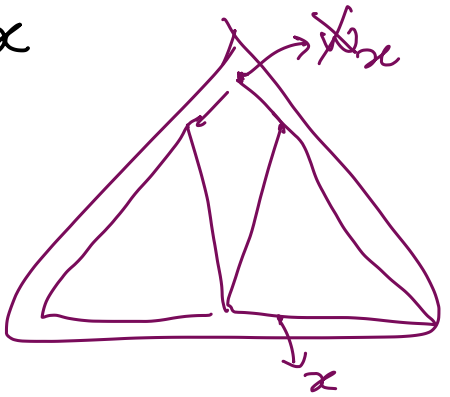
y's parent is outside the subtree rooted at u

$x \ge y_m$ o/w we wouldn't have swapped x, y_m.

$w_i \ge y_i$ o/w we would have swapped x, w_i instead of x, y_i

Extract Min ()

1. $w \leftarrow$ root.key
2. Let u be the rightmost node in the bottommost layer. Let x be the key of u.
   Disconnect u from its parent.
   (pop() in the array / list representation)
3. root.key $\leftarrow x$
4. Heap Down (root)
5. Return w.



Changekey (u, x) (u: node ; replace $u.key$ by x)

$x' \leftarrow u.key$

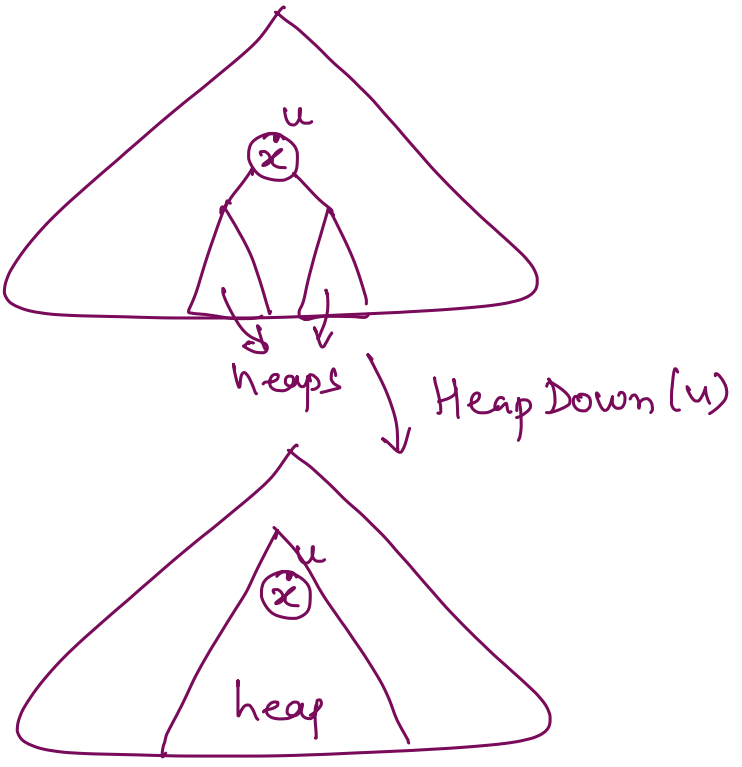
$u.key \leftarrow x$

If $x' > x$:

HeapUp(u)

If $x' < x$:

HeapDown(u)



Exercise: For the $x' < x$ case: prove that after HeapDown(u) the entire ACBT becomes a heap (not just the subtree rooted at u)

Build Heap (l) l: list of n keys

Construct a heap over the set of keys in list l.

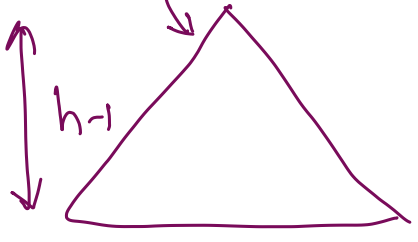
Obvious algorithm: Start with an empty heap; enqueue each element of l.

Running time: $O(n \log n)$

$l = [n, n-1, \underbrace{\ldots}_{2^{h-1}-1}, \underbrace{2^{h-1}, \ldots, 1}_{\approx \frac{n}{2}}]$

$n = 2^h - 1$

$h \approx \log_2 n$



$\Omega(h)$ insertion time for each of these

$\therefore$ The "Obvious algorithm" takes $\Theta(n \log n)$ time.

Next class: Faster Build Heap algorithm.